

Data Analytics Apps for a Smart Eco Gym

You have been hired by a next-generation, eco-friendly gym that is looking to incorporate new features to improve member experiences while increasing energy efficiency. The gym has collected a wide variety of sensor data and is seeking intuitive, user-friendly tools to help them make data-driven decisions.

In this project, you will design and build an interactive sensor monitoring dashboard in MATLAB that converts raw sensor data into clear, actionable metrics and insights. Your minimum viable product (MVP) should be able to process real-world data, compute meaningful metrics, and present results through an intuitive user interface.

Use this Live Script as a template to get started on your project solution. You may include helper functions at the bottom of this document or as separate files. For a tutorial on how to use Live Scripts, check out this [video](#).

Suggested Tasks

1. Define the problem and goal.
2. Explore and understand the data.
3. Process and prepare the data.
4. Define and test computing meaningful metrics.
5. Design and build your app.
6. Visualize and communicate results for your app user interface.
7. Evaluate and refine your solution.
8. Present your work.

Table of Contents

Task 1: Break down the problem.....	1
Task 2: Explore and understand the data.....	2
Task 2 Checkpoint.....	2
Task 3: Process and prepare the data.....	3
Task 3 Checkpoint.....	4
Task 4: Define and test computing meaningful metrics.....	5
Task 4 Checkpoint.....	6
Task 5: Design and build your MATLAB app.....	6
Task 5 Checkpoint.....	8
Task 6: Visualize and Communicate Results for your App UI.....	8
Task 6 Checkpoint.....	9
Task 7: Evaluate and refine your solution.....	9
Task 7 Checkpoint.....	11
Task 8: Present your work.....	11

Task 1: Break down the problem

In the space below, define the problem statement. Identify the specific question you want to answer:

Determine who your end user is and what decisions they need to make:

List the requirements, constraints, and success criteria for the project:

What is your proposed solution?

List the steps you will need to take to achieve your proposed solution:

What challenges do you face in the process of achieving the solution?

Task 2: Explore and understand the data

Before cleaning, processing, or building your app, take time to understand what data you have. The goal of this step is to identify the relevant files, variables, units, and patterns that will guide your metric and app design.

```
% Insert your code in the spaces below (or make use of helper functions and
% indicate where
% they can be found). Ensure your code is well-documented.

% STEP 1: Choose and load a relevant .mat file.
% Choose a data file or files that connect
% to your project question. Use the MATLAB 'load' function.

% STEP 2: Identify the important variables.
% Look at the fields in the data and
% identify which fields represent: time, sensor value,
% location/zone/equipment/person/session/design etc, labels or categories,
% units. Use the MATLAB 'fieldnames' function. Use the dataset README to
% confirm what each variable means and
% what units it uses.

% STEP 3: If helpful, convert the data to a table.

% STEP 4: Make at least one simple plot to look for patterns, trends, or
% unusual
% values.
% At this stage, your plot does not need to be polished, it's just
% a way to understand the data.
```

Task 2 Checkpoint

By the end of this step, you should be able to explain:

- Which .mat file(s) you loaded

- What the main variables mean
- What units are used
- How the data are organized
- Which variables will be useful for filtering or grouping
- What the data look like in at least one quick plot
- What patterns or unusual values you noticed
- Which data you plan to use in your app

Task 3. Process and prepare the data

After exploring the data, the next step is to prepare it for metric calculations and app visualizations. Processing helps turn raw sensor data into a cleaner, more useful form.

Start by choosing the main sensor signal or variable you want to analyze.

Examples include:

- heart rate
- acceleration
- air velocity
- power
- generated power
- CO2 concentration
- relative humidity
- TVOC concentration
- floor vibration
- occupancy count

You do not need to process every variable. Focus on the variables that connect most directly to your project goal.

Ask yourself:

- What sensor value matters most for my question?
- What variable will my metric depend on?
- What variable will I show in my app?
- Do I need to process one signal or compare multiple signals?

```
% Insert your code in the spaces below (or make use of helper functions and
% indicate where they can be found). Ensure your code is well-documented.
```

```
% STEP 1: Filter, group, or segment the data.
```

```
% Most datasets include more information than you need for one metric or
visualization.
```

```
% You might want to analyze only one gym zone or sensor location, compare
fan blade
```

```
% designs, or compare active vs idle equipment states (for example).

% STEP 2: Clean missing or unusual values
% Before computing metrics, check whether the data include missing or
unusual values.
% You may need to:
% remove missing values
% replace missing values
% ignore missing values in calculations
% identify unusually high or low values
% decide whether spikes are errors or meaningful events
% Helpful MATLAB functions include: isnan, ismissing, isoutlier, min, max,
% median, mean.

% STEP 3: Smooth, average, or summarize the data
% Raw sensor data can be noisy or too detailed for a dashboard. You may
need to simplify the data while keeping the important patterns.
% Common processing steps include:
% smoothing noisy signals
% averaging values over time
% calculating moving averages
% finding peaks
% calculating totals
% calculating minimum, maximum, or mean values
% summarizing values by zone, sensor, equipment, activity, or design
% Helpful MATLAB functions include: smoothdata, movmean, movmedian,
% findpeaks

% STEP 4: Prepare processed data for your app
% Before moving on to metric calculations, organize your processed data so
it will be easy to use in your app.
% Your processed data should be clearly named and organized so it's easier
% to plot and connect to your metric. You might want to create separate
% variables for raw, cleaned, filtered, summary data. This will make your
% code easier to debug and explain.
```

Task 3 Checkpoint

By the end of this step, you should be able to explain:

- Which signal or variable you processed
- Why that signal is important for your project goal
- How you filtered, grouped, or segmented the data
- How you handled missing or unusual values
- Whether you smoothed, averaged, summarized, or transformed the data
- What MATLAB functions you used
- What processed data will be used for your metric
- What processed data will be shown in your app

Task 4: Define and test computing meaningful metrics

After preparing your data, decide what metric or metrics will help your app user understand the system and make a decision.

First, choose a metric or metrics that connect to your project goal. Remember that these metrics should directly help answer your main question.

A good metric should be:

- easy to understand
- clearly connected to the data
- useful for decision-making
- explainable to a non-expert user

```
% Insert your code in the spaces below (or make use of helper functions and
% indicate where
% they can be found). Ensure your code is well-documented.
```

```
% STEP 1: Define how the metric(s) will be calculated
% Write/code the formula and logic. Your metric might be an average, a max
% or min, a total, a percentage, a score, a category, a comparison to a
% threshold, etc.
% Helpful MATLAB functions include: mean, max, min, sum, normalize
```

```
% STEP 2: Test the metric on a small subset of data
% Test your formula and logic on a small, simple part of the dataset. This
% helps you check your calculation works before applying it to the full
% dataset. Ask yourself if the result makes sense, if the units are
% correct, if the metric changes when the data change, and if it helps
% answer the main question.
```

```
% STEP 3: Compare the metric across groups or conditions.
```

```
% A metric becomes useful when users can compare it across meaningful
% categories. You might compare by zone, fan design, operating state,
% activity type, equipment type, etc.

% STEP 4: Decide how the metric should appear in your app
% Remember, your metric should the user move from data to insight to
% decision. How you display it to the user will help them interpret the
% metric. You might consider using: a single summary value, a color-coded
% indicator, a warning or alert, a bar chart, a trend line, a message, a
% heatmap, etc.
```

Task 4 Checkpoint

By the end of this step, you should be able to explain:

- What metric you chose
- Why the metric is useful for your project goal
- What data variables are needed to calculate it
- How the metric is calculated
- What units the metric uses, if any
- Whether the metric is a value, score, category, or index
- How you tested the metric on a small subset of data
- How the metric changes across groups or conditions
- How the metric will be displayed in your app
- What decision the metric could help your user make

Task 5: Design and build your MATLAB app

After choosing your data, processing steps, and metric, you are ready to design your MATLAB app. Your app should help a user interact with the data, understand the metric, and make a decision.

STEP 1: Before opening App Designer, sketch the basic workflow of your app.

A useful structure is:

Input → Processing → Metrics → Visualization → Decision

Your app should allow the user to:

- load or import data
- choose an input or setting
- run a calculation
- view a metric

- interpret the result using a graph or visual indicator

Ask yourself:

- What does the user need to select?
- What should happen after the user clicks a button or changes a dropdown?
- What metric should update?
- What graph should update?
- What decision should the app help the user make?

STEP 2: Choose your interactive components

Your app must include at least one interactive component.

Examples include:

- button
- dropdown menu
- slider
- checkbox
- text/number input

Choose interactive components that match your project goal. The interaction should change something meaningful in the app

```
% Insert your code in the spaces below (or make use of helper functions and
% indicate where
% they can be found). Ensure your code is well-documented.

% STEP 3: Organize your app code into different sections
% For example, these could be: data loading, data processing, metric
% calculation, visualization, and users interaction callbacks.
% Tip: try to avoid putting all your code inside one callback function.
% Instead, create helper functions when possible.
% Organize your app so that each part of the workflow has a clear purpose.
% This makes your app easier to debug, explain, and improve.
```

STEP 4: Use MATLAB App Designer to create your user interface.

Your app should include:

- a clear title
- a way to load or select data
- at least one interactive component
- at least one graph
- at least one visual indicator

- labels and units
- a layout that is easy to understand

A good app should feel like a usable decision-making tool, not just a static plot.

Task 5 Checkpoint

By the end of this step, you should be able to explain:

- What your app user can do
- What .mat file your app imports
- What interactive component your app includes
- What happens when the user interacts with the app
- What metric your app calculates
- What data processing step your app performs
- What graph your app displays
- What visual indicator your app displays
- How your code is organized
- How your app helps the user make a decision

Task 6: Visualize and Communicate Results for your App UI

After your app loads data, processes it, and computes metrics, you need to decide how to communicate the results clearly. Your app should help a non-expert user quickly understand what is happening and what decision they might make.

Remember to focus on insight, not raw data. Use your UI to guide the user from:

Data → Metric → Meaning → Decision

For each graph or visual element, ask:

- What should the user notice?
- What does this metric mean?
- Is this result good, bad, normal, or unusual?
- What action might the user take?
- Is there a simpler way to show this result?

% Insert your code in the spaces below (or make use of helper functions and indicate where

% they can be found). Ensure your code is well-documented.

% STEP 1: Choose a graph or display that matches the type of result you want to communicate. For example, for a spatial pattern, you might use a heatmap. For an alert, you might use a color-coded lamp. For change over time, you might use a time-series plot.

% Helpful MATLAB functions include: plot, scatter, bar, imagesc, heatmap, histogram.


```
% STEP 2: Label everything clearly
% Make sure your user can clearly understand the visuals. Include graph
% titles, axis labels, units, legends, colorbar labels, short explanatory
% text, clear names for zones/sensors/designs/activity/equipment, etc.
% Helpful MATLAB functions include: title, xlabel, ylabel, legend,
% colorbar, xlim, ylim
```

```
% STEP 3: Make sure visuals respond to user interaction. Your plots and
% indicators should update when the user interacts with the app. If an
% input changes, the app should update.
```

Task 6 Checkpoint

By the end of this step, you should be able to explain:

- What graph or visualization your app includes
- Why that visualization matches your data and metric
- What labels, units, and titles you added
- What visual indicator your app includes
- How the visual indicator helps the user interpret the result
- What insight the user should take away
- What decision the visualization could support
- How the visualization updates when the user interacts with the app
- How your UI avoids overwhelming the user with raw data

Task 7: Evaluate and refine your solution

After building your app, test whether it actually answers your project question and helps the user make sense of the data. Evaluation is not just checking whether the app runs. It is checking whether the app is useful, clear, and reliable.

STEP 1: After building your app, test it carefully. Try changing (for example, depending on your mini design challenge):

- selected zone
- selected sensor
- selected equipment
- selected threshold
- selected time range

- selected metric
- imported data file

Check whether:

- the results make sense
- the app loads the data correctly
- the metric updates correctly
- the graph updates correctly
- the visual indicator updates correctly
- labels and units are clear
- the app gives useful information to the user
- the app still works if the user makes a different selection

STEP 2: Evaluate the user experience

Your app should be understandable to someone who did not write the code.

Ask:

- Is the purpose of the app clear?
- Is it obvious what the user should click or select?
- Are the graph labels and units clear?
- Are the visual indicators easy to interpret?
- Does the app avoid showing too much raw data?
- Does the app explain what the metric means?
- Does the app help the user make a decision?

STEP 3: Identify design tradeoffs and limitations

Every engineering solution involves tradeoffs. Identify what choices you made and what limitations your app has.

Examples of tradeoffs include:

- using a simple metric instead of a more complex one
- smoothing data to show trends but reducing short-term detail
- using a threshold that is easy to interpret but somewhat simplified
- focusing on one zone, sensor, or system instead of the entire dataset
- prioritizing a clean user interface over showing every possible data field

Examples of limitations include:

- the data are synthetic
- the app uses simplified assumptions
- the metric may not capture every important factor
- the alert threshold may not represent a professional standard

- the app may only work with one dataset format
- the app may not include every possible user scenario

Your goal is not to create a perfect app. Your goal is to understand and explain the choices you made.

STEP 4: Refine the app based on testing

Use what you learned from testing to improve your app.

You might revise:

- metric formulas
- data processing steps
- graph type
- labels or units
- color indicators
- alert messages
- layout
- user interaction options
- code organization

After making changes, test the app again.

Task 7 Checkpoint

By the end of this step, you should be able to explain:

- How you tested your app
- What inputs, scenarios, or selections you tested
- Whether the app results made sense
- Whether the graph and visual indicator were clear
- Whether the app answered the original project question
- What tradeoffs you made
- What limitations your solution has
- What changes you made after testing
- What you would improve with more time

Task 8: Present your work

After building, testing, and refining your app, prepare your final project materials so that someone else can understand what you built, how it works, and why it matters.

STEP 1: Prepare your code for submission as a public GitHub repository.

- Follow [these guidelines](#) for your repository
- New to GitHub? Check out these [introductory resources](#)

Your repository should include:

- your MATLAB app file (.mlapp)
- any supporting MATLAB functions or scripts (.m)
- a descriptive README.md file
- any required data files or clear instructions for where to find them
- clear instructions for opening and running your app

Your README should explain your project clearly enough that a new user can download, run, and understand your app, including:

- the mini challenge your team addressed
- the problem or question your app investigates
- who the intended user is
- what data files your app uses
- what metric or metrics your app computes
- what data processing steps your app performs
- what interactive features your app includes
- what graphs or visual indicators your app displays
- how to run the app
- any assumptions, limitations, or known issues

Good documentation helps others understand both your code and your design choices

STEP 2: Prepare a presentation or report

Prepare either a presentation or a written report that clearly communicates your project.

Your presentation or report should include:

- your project goal and approach
- the data you used
- your key design decisions
- important tradeoffs you considered
- your selected metric or metrics
- your data processing steps
- a demonstration or walkthrough of your app
- key findings or insights
- potential next steps or improvements

When presenting your app, focus on how it helps the user make sense of the data. Show how your app connects:

Data → Metrics → Insights → Decisions

End by discussing how your app could be improved. Possible next steps might include:

- adding more user interaction

- improving the metric
- improving the visual design
- adding more datasets
- testing with more scenarios
- making the app easier for a non-expert user
- adding clearer alerts or recommendations
- improving code organization or documentation

Your final presentation or report should show not only what you built, but how your engineering decisions shaped the final solution. Include the presentation or report in your GitHub repository.